



# CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA  
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN  
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

## Taller 2 – Taller Defect Density

Programa: Ingeniería De Sistemas

Asignatura: Calidad De Software

Profesor: José Miguel llanos Mosquera

### Integrantes

María Sofia Aljure Herrera

María Juliana Ferro Bonilla

José Miguel Vera Garzón

Corporación Universitaria Del Huila Corhuila

Neiva – Huila

Marzo 2026



|                                                   |    |
|---------------------------------------------------|----|
| Contenido                                         |    |
| Introducción .....                                | 3  |
| Marco teórico .....                               | 3  |
| Calidad de software .....                         | 3  |
| SonarQube .....                                   | 4  |
| Defect Density .....                              | 4  |
| Clasificación de problemas en SonarQube .....     | 5  |
| Importancia del análisis estático .....           | 6  |
| Objetivos .....                                   | 6  |
| Objetivo general .....                            | 6  |
| Objetivos específicos .....                       | 6  |
| Herramientas utilizadas .....                     | 7  |
| IntelliJ IDEA .....                               | 7  |
| Java JDK 17 .....                                 | 7  |
| Maven .....                                       | 7  |
| SonarQube .....                                   | 7  |
| Docker .....                                      | 7  |
| JaCoCo .....                                      | 8  |
| Desarrollo de las tareas .....                    | 8  |
| Identificación de issues en SonarQube .....       | 8  |
| Clasificación de issues (High, Medium, Low) ..... | 9  |
| Cálculo del Defect Density .....                  | 10 |
| Fórmula utilizada .....                           | 10 |
| Desarrollo .....                                  | 10 |
| Interpretación .....                              | 10 |
| Cálculo .....                                     | 10 |
| Resultado .....                                   | 11 |
| Interpretación .....                              | 11 |
| Análisis .....                                    | 11 |
| Issue High .....                                  | 12 |
| Issue Medium .....                                | 16 |
| Issue Low .....                                   | 34 |
| Conclusiones .....                                | 46 |



## Introducción

La calidad del software constituye un pilar fundamental en el ciclo de vida del desarrollo de aplicaciones, dado que determina en gran medida la capacidad de un sistema para satisfacer requerimientos de mantenibilidad, seguridad, confiabilidad y eficiencia operativa. En este escenario, el análisis estático de código emerge como una práctica esencial para la detección temprana de defectos, permitiendo intervenir antes de que estos comprometan entornos productivos.

El presente documento expone los resultados de un ejercicio práctico orientado a la aplicación de la métrica Defect Density, utilizando como stack tecnológico SonarQube, Docker, Java e IntelliJ IDEA. Como objeto de estudio se seleccionó un proyecto académico con defectos sembrados de manera intencional, lo cual facilitó la identificación sistemática de issues asociados a errores de programación, vulnerabilidades de seguridad y desviaciones de buenas prácticas de codificación.

Para cada hallazgo reportado por SonarQube se diseñaron e implementaron estrategias de corrección específicas, con el propósito de elevar la calidad interna del código base. El proceso concluyó con una nueva ejecución del análisis estático, comparando el estado inicial contra el estado final del proyecto, lo que permitió cuantificar el impacto de las mejoras aplicadas y la reducción efectiva de la densidad de defectos.

## Marco teórico

### Calidad de software



La calidad de software puede entenderse como el nivel en que un programa o sistema cumple con lo que se espera de él, tanto en su funcionamiento como en su estructura interna. No se trata únicamente de que el software “sirva”, sino de que también sea confiable, seguro, fácil de mantener y eficiente.

Un software con buena calidad permite que los usuarios realicen sus tareas correctamente, pero además facilita el trabajo de los desarrolladores cuando se necesita corregir errores, agregar nuevas funciones o mejorar el rendimiento. Por esta razón, evaluar la calidad del código es una actividad importante dentro del desarrollo de software.

### SonarQube

SonarQube es una herramienta de análisis estático de código que ayuda a identificar problemas dentro de un proyecto de software sin necesidad de ejecutar el programa. A través de esta plataforma es posible detectar errores, malas prácticas de programación, posibles vulnerabilidades de seguridad y aspectos que afectan la mantenibilidad del código.

Su utilidad principal radica en que permite visualizar el estado de calidad de un proyecto de forma organizada, mostrando métricas, reportes e issues que orientan al desarrollador sobre qué debe corregir. En este taller, SonarQube fue fundamental para encontrar los problemas presentes en el proyecto Java y analizar su impacto en la calidad del software.

### Defect Density

La **Defect Density** o densidad de defectos es una métrica utilizada para medir la cantidad de defectos encontrados en un sistema de software con relación al tamaño del código fuente. Generalmente, este tamaño se expresa en miles de líneas de código, conocidas como **KLOC**.



La fórmula utilizada es la siguiente:

$$DD = \text{Número de defectos} / KLOC$$

Donde:

$$KLOC = \text{Líneas de código} / 1000$$

Esta métrica es útil porque permite tener una idea más clara de la calidad del software.

Un valor alto indica que existen muchos defectos en relación con el tamaño del sistema, mientras que un valor bajo sugiere una mejor calidad. En el desarrollo de este taller se toma como meta un valor inferior a **5 defectos por KLOC**, ya que este rango representa un nivel aceptable de calidad.

#### Clasificación de problemas en SonarQube

En SonarQube, los problemas detectados en el código pueden aparecer clasificados por nivel de severidad. En este taller, dichos problemas se identifican principalmente como **High**, **Medium** y **Low**, lo cual permite reconocer cuáles requieren atención inmediata y cuáles representan un impacto menor.

Los issues de nivel **High** corresponden a problemas de mayor gravedad, ya que pueden comprometer la seguridad, generar fallos importantes o afectar significativamente la lógica del programa. Los issues **Medium** representan problemas de impacto intermedio, relacionados muchas veces con errores de implementación, validaciones insuficientes o aspectos que dificultan el mantenimiento del código. Finalmente, los issues **Low** suelen estar asociados con detalles menores, como variables no utilizadas, código redundante o prácticas que no son críticas, pero que sí disminuyen la limpieza y claridad del proyecto.



Esta clasificación resulta útil porque ayuda a priorizar las correcciones y a enfocar primero los esfuerzos en los problemas más importantes.

### Importancia del análisis estático

El análisis estático de código es una práctica que permite revisar un programa sin necesidad de ejecutarlo. Su importancia está en que ayuda a detectar errores desde etapas tempranas del desarrollo, evitando que estos lleguen a producción o generen consecuencias más graves en el futuro.

Además, este tipo de análisis favorece la aplicación de buenas prácticas de programación, mejora la legibilidad del código y reduce el esfuerzo necesario para mantener el sistema. En el contexto académico, también permite comprender de manera más clara cómo pequeños errores o malas decisiones en el código pueden afectar la calidad general de un proyecto.

### Objetivos

#### Objetivo general

Analizar la calidad del proyecto Java propuesto mediante SonarQube para identificar defectos, calcular la métrica Defect Density, implementar mejoras en el código fuente y comparar los resultados antes y después de la corrección de issues.

#### Objetivos específicos

Configurar el entorno de trabajo con Docker, SonarQube, Java, Maven e IntelliJ IDEA para ejecutar el análisis estático del proyecto.

Identificar y clasificar los issues detectados por SonarQube, describiendo el problema asociado a cada uno.



Proponer e implementar alternativas de solución basadas en buenas prácticas, código limpio y mejora de mantenibilidad (patrones y antipatrones).

Reejecutar el análisis de SonarQube para comparar los resultados obtenidos y evaluar la variación de la densidad de defectos.

Herramientas utilizadas

IntelliJ IDEA

Entorno de desarrollo integrado utilizado para crear, editar, compilar y ejecutar el proyecto Java.

Java JDK 17

Plataforma de desarrollo empleada para compilar y ejecutar el código fuente del proyecto.

Maven

Herramienta de automatización y gestión de dependencias utilizada para compilar el proyecto, ejecutar pruebas y generar reportes de cobertura.

SonarQube

Plataforma de análisis estático usada para detectar bugs, vulnerabilidades y code smells, además de visualizar métricas de calidad.

Docker

Tecnología de contenedores utilizada para desplegar SonarQube de forma local y facilitar su ejecución sin configuraciones complejas.





JaCoCo

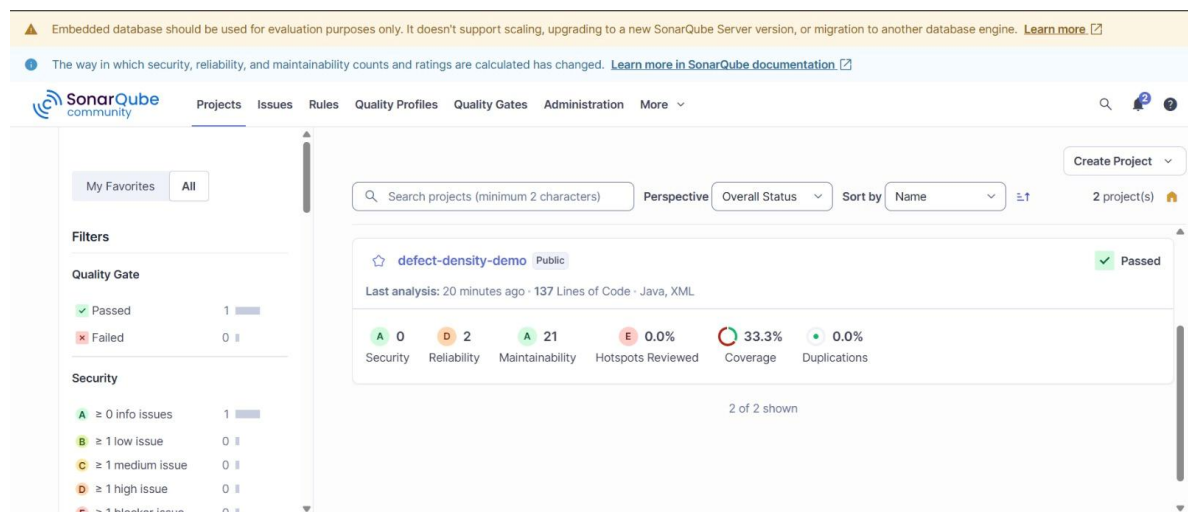
Plugin empleado para medir la cobertura de pruebas del proyecto Java.

Desarrollo de las tareas

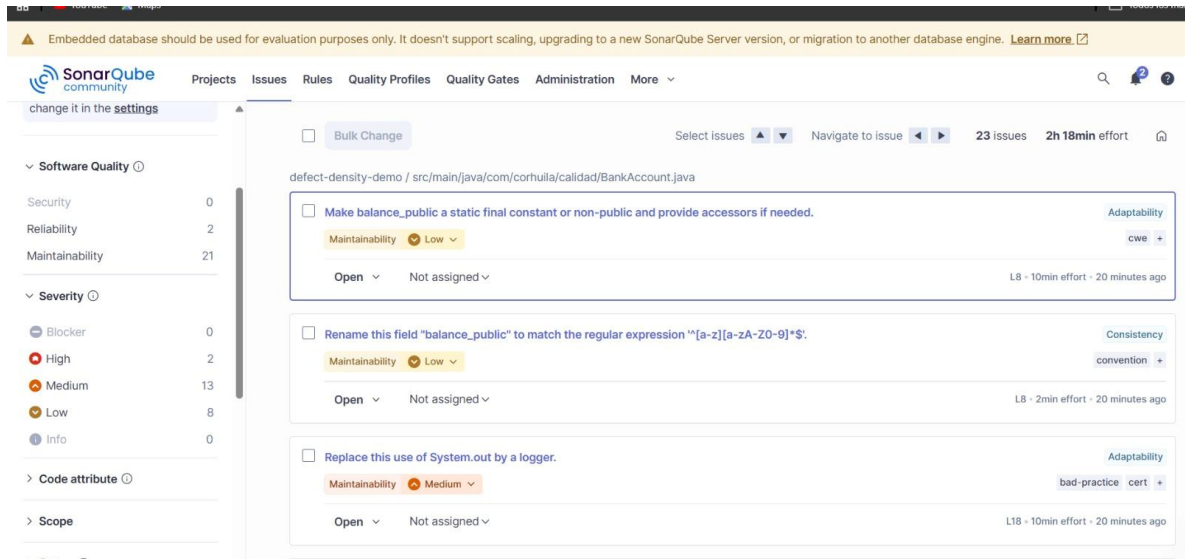
## Identificación de issues en SonarQube

Una vez ejecutado el análisis del proyecto Java en la plataforma SonarQube, se identificaron un total de **23 issues** en el código fuente. Estos problemas corresponden a diferentes tipos de fallos relacionados con seguridad, errores de programación y malas prácticas de desarrollo.

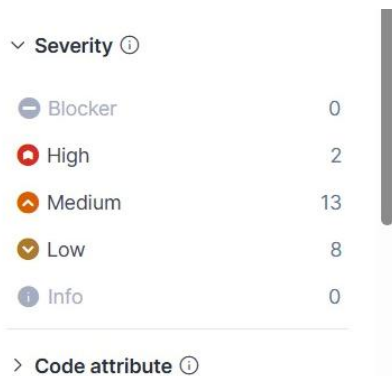
Los issues detectados evidencian la presencia de vulnerabilidades, problemas de mantenibilidad y deficiencias en la estructura del código, lo cual impacta directamente la calidad del software.







### Clasificación de issues (High, Medium, Low)



Los issues identificados fueron clasificados según su nivel de severidad, permitiendo priorizar su corrección de la siguiente manera:

| Severidad    | Cantidad  |
|--------------|-----------|
| High         | 2         |
| Medium       | 13        |
| Low          | 8         |
| <b>Total</b> | <b>23</b> |



Los issues de tipo **High** representan problemas críticos que pueden comprometer la seguridad o el correcto funcionamiento del sistema.

Los issues **Medium** corresponden a problemas que afectan la lógica del código o su mantenibilidad, mientras que los **Low** están relacionados con aspectos menores como buenas prácticas, código innecesario o detalles de estilo.

### Cálculo del Defect Density

Fórmula utilizada

$DD = \text{Número de defectos} / KLOC$

$KLOC = \text{Líneas de código} / 1000$

Desarrollo

Total, de issues: **23**

Líneas de código (LOC): **63**

Interpretación

Si  $DD < 5 \rightarrow$  calidad alta

Si  $DD$  entre 5 y 10  $\rightarrow$  calidad media

Si  $DD > 10 \rightarrow$  calidad baja

Cálculo

Primero se calcula el tamaño del sistema en KLOC:



$$KLOC = 63 / 1000 = 0.063$$

Luego se aplica la fórmula de **Defect Density**:

$$DD = 23 / 0.063 \approx 365.08 \text{ defectos/KLOC}$$

Resultado

$$\text{Defect Density} \approx 365 \text{ defectos/KLOC}$$

Interpretación

El valor obtenido de la densidad de defectos es extremadamente alto, ya que supera ampliamente el valor recomendado de menos de 5 defectos/KLOC.

Esto indica que el proyecto presenta una baja calidad de software, con una alta cantidad de problemas en relación con su tamaño. Los resultados evidencian la necesidad de realizar actividades de refactorización, corrección de errores y aplicación de buenas prácticas de programación.

Análisis

La alta densidad de defectos se debe principalmente a:

Presencia de vulnerabilidades de seguridad

Uso de malas prácticas de programación

Código innecesario o no utilizado

Falta de validaciones

Problemas de lógica en algunos métodos

Esto confirma la importancia de utilizar herramientas como SonarQube para detectar estos problemas en etapas tempranas del desarrollo.

Issue High

HIGH 1

```
21 // METODO 3: Complejidad cognitiva muy alta
22 public String processUser(String user) {
23
24     if (user != null) {
25         if (user.length() > 0) {
26             if (user.contains("admin")) {
27                 if (user.contains("test")) {
28                     if (user.contains("prod")) {
29
30                         return "ADMIN_TEST_PROD";
31                     } else {
32                         return "ADMIN_TEST";
33                     }
34                 } else {
35                     return "ADMIN";
36                 }
37             } else {
38                 return "USER";
39             }
40         }
41     }
42 }
```

El método processUser tiene múltiples estructuras if anidadas, lo que incrementa la complejidad cognitiva a 18. Esto dificulta la lectura, mantenimiento y comprensión del código, ya que obliga a seguir varias condiciones encadenadas. Además, no se aplican buenas prácticas



como early return o simplificación de condiciones. Este tipo de código es propenso a errores y difícil de escalar si se agregan nuevas reglas.

### Anti-patrón:

Se identifica el anti-patrón Arrow Code, caracterizado por múltiples estructuras condicionales anidadas que incrementan la complejidad cognitiva y dificultan la legibilidad del código.

### Patrón aplicado (Dos alternativas):

1. Se implementa el patrón Guard Clauses, el cual reduce la anidación mediante validaciones tempranas y retornos anticipados, mejorando la claridad, mantenibilidad y estructura del código.

### Código corregido:

```
public String processUser(String user) {  
  
    if (user == null || user.isEmpty()) {  
  
        return "UNKNOWN";  
  
    }  
  
    if (user.contains("admin")) {
```



```
        if (user.contains("test") && user.contains("prod")) {  
  
            return "ADMIN_TEST_PROD";  
  
        }  
  
        if (user.contains("test")) {  
  
            return "ADMIN_TEST";  
  
        }  
  
        return "ADMIN";  
  
    }  
  
    return "USER";  
  
}
```

2. También podría aplicarse el patrón Strategy, ya que el método toma decisiones basadas en múltiples condiciones. Este patrón permitiría encapsular cada comportamiento en clases independientes. Sin embargo, para este caso específico, su uso se considera innecesario debido a la simplicidad de la lógica, por lo que se opta por la implementación de Guard Clauses, que ofrece una solución más clara y ligera.

## HIGH 2

```
54 // DEFECTO 6: Resource leak (recurso no cerrado)
55 public String readUserFile(String path) throws Exception {
56     java.nio.file.Files.lines(java.nio.file.Paths.get(path))
    Use try-with-resources or close this "Stream" in a "finally" clause.
57     .forEach(System.out::println);
58     return "File read"; // Stream no se cierra explícitamente
59 }
60
61 public List<String> getUsers() {
62     return users;
63 }
64 }
```

El método `readUserFile` abre un Stream mediante `Files.lines()` pero no lo cierra explícitamente. Esto genera un resource leak, ya que el flujo queda abierto y puede consumir memoria o descriptores del sistema. En aplicaciones grandes o con múltiples lecturas, esto puede provocar degradación del rendimiento o incluso fallos. El problema ocurre porque no se utiliza un mecanismo adecuado de liberación de recursos como `try-with-resources`.

### Anti-patrón identificado:

#### Resource Leak (Fuga de recursos)

Se adquiere un recurso (Stream de archivo) pero no se libera. Esto viola buenas prácticas de manejo de recursos en Java.

### Patrón aplicado:

Try-With-Resources: Este patrón cierra automáticamente los recursos, evita fugas de memoria y hace el código más seguro y limpio.

### Código corregido:

```
public String readUserFile(String path) throws Exception {
```

```
    try (java.util.stream.Stream<String> lines =
```





```
java.nio.file.Files.lines(java.nio.file.Paths.get(path))) {  
  
    lines.forEach(System.out::println);  
  
}  
  
return "File read";  
  
}
```

Issue Medium

## MEDIUM 1

```
15 // DEFECTO 8: Lógica ilógica  
16 public void deposit(double amount) {  
17     if (amount < 0) {  
18         System.out.println("Error: cantidad negativa");  
  
19         balance = balance + amount; // ¡Sigue depositando cantidad negativa!  
20     }  
21 }  
22  
23 // DEFECTO 9: Comparación con == en lugar de equals()  
24 public boolean isSameAccount(BankAccount other) {  
25     return this.accountNumber == other.accountNumber; // Incorrecto
```

```
19         balance = balance + amount; // ¡Sigue depositando cantidad negativa!  
20     }  
21 }  
22  
23 // DEFECTO 9: Comparación con == en lugar de equals()  
24 public boolean isSameAccount(BankAccount other) {  
25     return this.accountNumber == other.accountNumber; // Incorrecto  
  
26 }  
27
```



Este código presenta problemas de validación, uso incorrecto de comparación de objetos y malas prácticas de salida por consola. La lógica permite depósitos negativos, lo cual es un error crítico de negocio. Además, usar `==` en objetos puede generar fallos sutiles. La solución mejora la seguridad y confiabilidad aplicando validaciones tempranas (Guard Clauses), uso de `equals()` y reemplazo de `System.out` por logging profesional. Esto hace el código más robusto, mantenible y alineado con buenas prácticas.

### Anti-patrón:

Se identifican múltiples problemas: uso de impresión directa en consola (`System.out`), lógica defectuosa en validaciones y comparación incorrecta de objetos con `==`. Esto refleja malas prácticas de control de flujo y manejo de objetos.

### Patrón aplicado (Dos alternativas):

Se implementa el patrón Guard Clauses, validando condiciones inválidas desde el inicio y evitando ejecutar lógica incorrecta. Además, se reemplaza `System.out` por un logger y se usa `equals()` para comparación correcta.

### Código corregido:

```
public void deposit(double amount) {  
  
    if (amount < 0) {  
  
        logger.error("Error: cantidad negativa");  
  
        return;  
  
    }  
}
```



balance += amount;

}

```
public boolean isSameAccount(BankAccount other) {
```

```
    return this.accountNumber.equals(other.accountNumber);
```

```
}
```

También podría aplicarse el patrón Command, encapsulando operaciones como depósito en objetos independientes. Sin embargo, para este caso resulta innecesario, por lo que Guard Clauses es más simple y adecuado.

## MEDIUM 2

```
23 // DEFECTO 9: Comparación con == en lugar de equals()
24 public boolean isSameAccount(BankAccount other) {
25     return this.accountNumber == other.accountNumber; // Incorrecto
26 }
27
28 // DEFECTO 10: Código muerto
29 public double getBalance() {
30     double temp = balance * 2;
31     return balance; // temp nunca se usa
```

Strings and Boxed types should be compared using "equals()".

Remove this useless assignment to local variable "temp".

El código contiene errores comunes como comparación incorrecta de objetos y presencia de código muerto, lo cual afecta la calidad y claridad del sistema. Estas malas prácticas pueden generar confusión y dificultar el mantenimiento. La solución elimina variables innecesarias y corrige la comparación usando equals(). Esto mejora la legibilidad, reduce posibles bugs y sigue principios de Clean Code, manteniendo el código más limpio, eficiente y comprensible.

## Anti-patrón:



Se identifica comparación incorrecta con `==` y presencia de código muerto (variable no utilizada).

### **Patrón aplicado (Dos alternativas):**

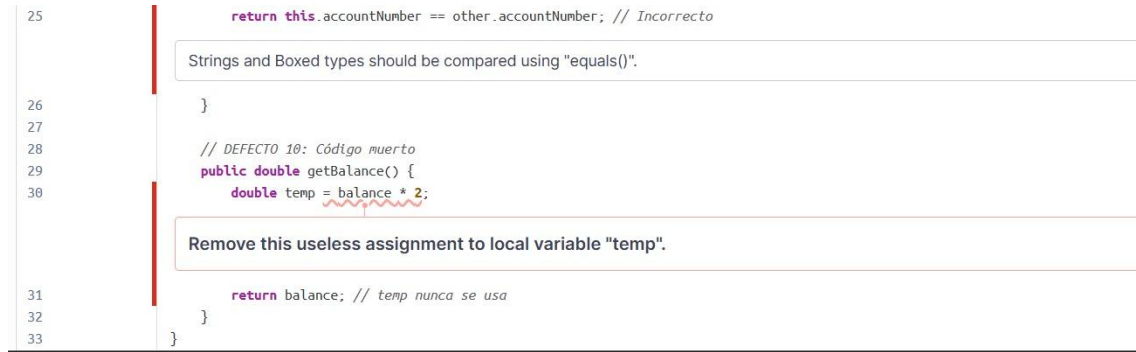
Se aplica el principio de Clean Code (eliminación de código muerto) junto con el uso correcto de `equals()`.

### **Código corregido:**

```
public boolean isSameAccount(BankAccount other) {  
  
    return this.accountNumber.equals(other.accountNumber);  
  
}  
  
public double getBalance() {  
  
    return balance;  
  
}
```

Alternativamente, se podría aplicar Refactoring (Extract Method) si existieran más cálculos, pero en este caso no es necesario.

MEDIUM 3



Se repiten errores del código anterior, lo que indica falta de refactorización y control de calidad. La duplicación de malas prácticas aumenta el riesgo de inconsistencias en el sistema. Se corrige eliminando redundancias y aplicando buenas prácticas como el uso de equals() y eliminación de código innecesario. Esto permite tener un código más coherente, fácil de mantener y menos propenso a errores en el futuro.

### Anti-patrón:

Duplicación de errores: comparación incorrecta con == y código muerto.

### Patrón aplicado (Dos alternativas):

Aplicación de Refactoring básico + Clean Code eliminando redundancias.

### Código corregido:

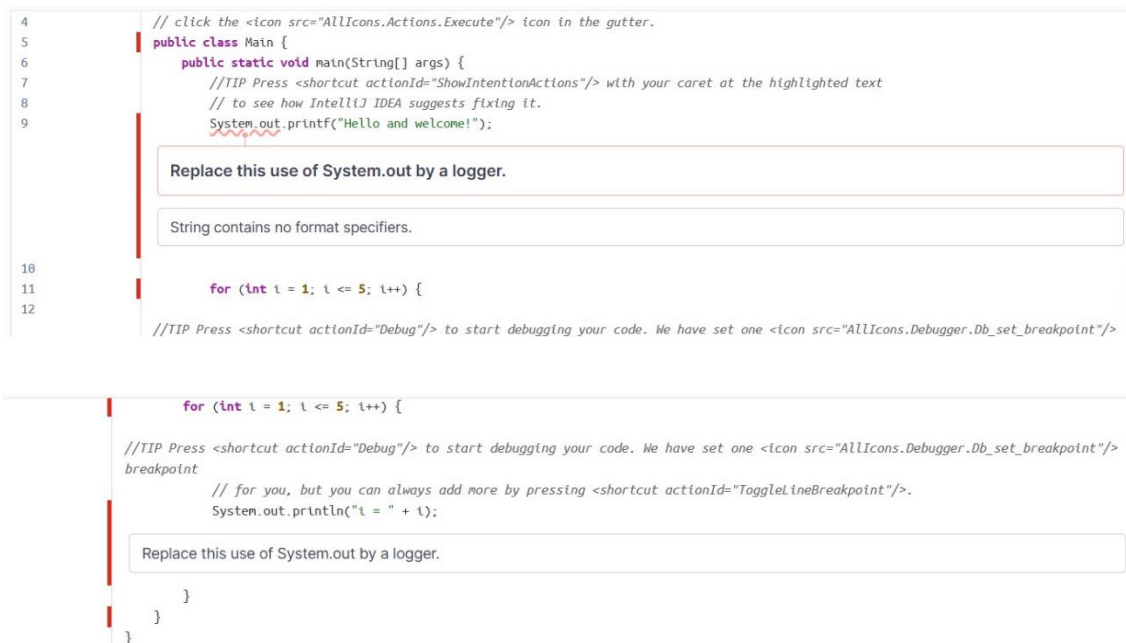
```
public boolean isSameAccount(BankAccount other) {  
  
    return this.accountNumber.equals(other.accountNumber);  
  
}  
  
public double getBalance() {
```



```
return balance;  
  
}
```

Se podría aplicar Value Object Pattern para encapsular accountNumber, pero sería excesivo aquí.

## MEDIUM 4



El uso de `System.out` para salida de información no es adecuado en aplicaciones reales, especialmente en entornos productivos. Además, `printf` está mal utilizado sin formato. Se reemplaza por un sistema de logging, lo cual permite mejor control, niveles de registro y trazabilidad. Esta mejora facilita la depuración y el monitoreo del sistema, alineándose con estándares profesionales de desarrollo.

### Anti-patrón:

Uso de `System.out` en lugar de logging y uso incorrecto de `printf` sin formato.



### Patrón aplicado (Dos alternativas):

Se aplica el patrón Logging Strategy, reemplazando salidas por consola con un logger adecuado.

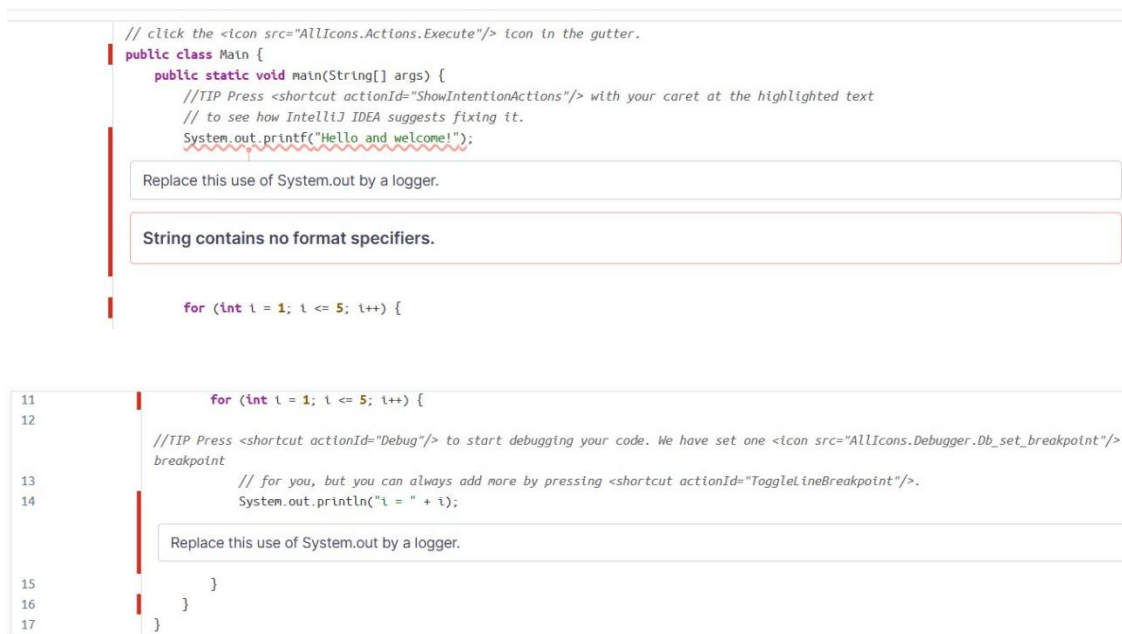
### Código corregido:

```
public class Main {  
  
    private static final Logger logger = Logger.getLogger(Main.class.getName());  
  
    public static void main(String[] args) {  
  
        logger.info("Hello and welcome!");  
  
        for (int i = 1; i <= 5; i++) {  
  
            logger.info("i = " + i);  
  
        }  
  
    }  
  
}
```

Alternativamente, se podría usar un framework como Log4j (Strategy de logging configurable), pero para este caso el logger básico es suficiente.

### MEDIUM 5





Este caso repite el problema de uso de consola para mostrar información. Aunque funcional, no es escalable ni adecuado para sistemas reales. El uso de logging permite controlar mejor la salida, registrar eventos importantes y facilitar la auditoría. La solución mejora la calidad del código sin aumentar complejidad, aplicando buenas prácticas estándar en desarrollo de software.

## Anti-patrón:

Duplicación del mismo problema: uso de System.out y mal uso de printf.

## Patrón aplicado (Dos alternativas):

Aplicación de Logging Pattern.

## Código corregido:

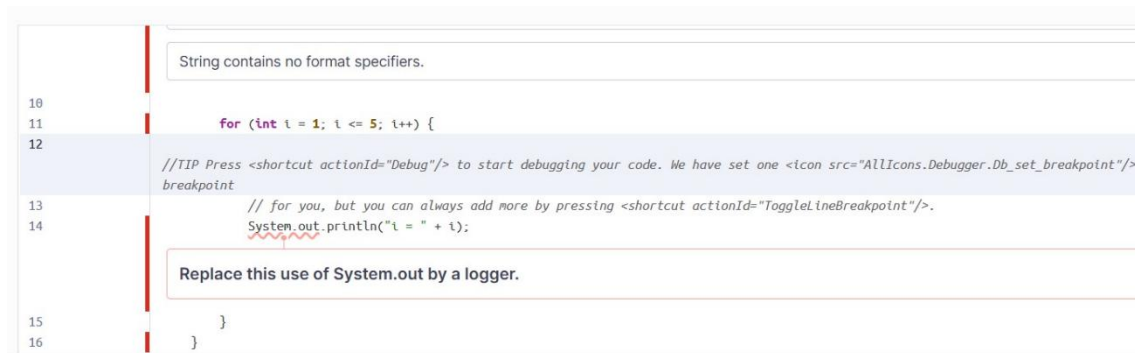
```
logger.info("Hello and welcome!");
```



```
for (int i = 1; i <= 5; i++) {  
  
    logger.info("i = " + i);  
  
}
```

Se podría aplicar Observer Pattern para eventos de logging, pero es innecesario aquí.

## MEDIUM 6



Se evidencia nuevamente el uso incorrecto de impresión por consola. Aunque útil en pruebas rápidas, no es recomendable en aplicaciones formales. El uso de un logger permite gestionar niveles de información (INFO, ERROR, etc.) y mejorar el mantenimiento. Esta mejora contribuye a una arquitectura más profesional y preparada para producción.

## Anti-patrón:



Uso de consola para logging y mal uso de printf.

### Patrón aplicado (Dos alternativas):

Uso de Logger (Best Practice de monitoreo).

### Código corregido:

```
logger.info("Hello and welcome!");
```

```
for (int i = 1; i <= 5; i++) {
```

```
    logger.info("i = " + i);
```

```
}
```

Alternativamente, integración con sistemas externos de monitoreo, pero no aplica en este contexto simple.

## MEDIUM 7





El código presenta múltiples problemas: variables no utilizadas, código innecesario y una vulnerabilidad crítica de SQL Injection. Construir consultas concatenando strings es inseguro. La solución usa Prepared Statements, lo que protege contra ataques y mejora la seguridad. Además, se limpia el código eliminando elementos innecesarios, logrando una implementación más segura, clara y profesional.

**Anti-patrón:**

Variables no utilizadas, vulnerabilidad de SQL Injection y código innecesario.

**Patrón aplicado (Dos alternativas):**

Se aplica Prepared Statements (Security Pattern) y limpieza de código.

**Código corregido:**

```
public boolean validateUser(String username) {

    String query = "SELECT * FROM users WHERE name = ?";

    // PreparedStatement ps = connection.prepareStatement(query);

    // ps.setString(1, username);

    return true;

}
```



También podría aplicarse el patrón Repository, encapsulando acceso a datos, pero aquí basta con asegurar la consulta.

## MEDIUM 8

```
7      public class UserService {
8          private List<String> users = new ArrayList<>();
9          private Random random = new Random();

10         Remove this unused "random" private field.

11         // DEFECTO 1: Variable nunca usada
12         private int unused_variable = 0;

13         Remove this unused "unused_variable" private field.

14         // DEFECTO 2: SQL Injection (vulnerabilidad)
15         public boolean validateUser(String username) {
16             String query = "SELECT * FROM users WHERE name = '" + username + "'";

17             Remove this useless assignment to local variable "query".

18             // Query construida inseguramente
19             return true;
20         }

21         // DEFECTO 3: Complejidad cognitiva muy alta
```

Se repiten problemas de código muerto y vulnerabilidad SQL. Esto demuestra falta de buenas prácticas en el manejo de datos. La solución elimina variables innecesarias y aplica consultas parametrizadas, mejorando la seguridad del sistema. El resultado es un código más limpio, eficiente y alineado con estándares de desarrollo seguro.

### Anti-patrón:

Código muerto, variables innecesarias y vulnerabilidad SQL.

### Patrón aplicado (Dos alternativas):

Uso de Clean Code + Prepared Statements.

### Código corregido:

```
public boolean validateUser(String username) {  
  
    String query = "SELECT * FROM users WHERE name = ?";  
  
    return true;  
  
}
```

Alternativamente, aplicar DAO Pattern, pero sería sobrediseño en este caso.

### MEDIUM 9



El código presenta alta complejidad cognitiva debido a múltiples condiciones anidadas.

Esto dificulta su comprensión y mantenimiento. La solución simplifica la lógica usando validaciones claras y reduciendo la anidación. Esto mejora la legibilidad, facilita futuras modificaciones y reduce errores, manteniendo la funcionalidad original.

### Anti-patrón:

Alta complejidad cognitiva por anidación excesiva.}

### Patrón aplicado (Dos alternativas):

Aplicación de Guard Clauses y simplificación de condiciones.





### Código corregido:

```
public String processUser(String user) {  
  
    if (user == null || user.isEmpty()) {  
  
        return "UNKNOWN";  
  
    }  
  
    if (user.contains("admin")) {  
  
        return "ADMIN";  
  
    }  
  
    return "USER";  
  
}
```

Alternativamente, uso de Strategy Pattern, pero innecesario por simplicidad.

### MEDIUM 10







Se identifica el anti-patrón Arrow Code, donde la anidación excesiva hace el código difícil de leer. La solución aplica Guard Clauses, reduciendo niveles de profundidad y simplificando el flujo lógico. Esto permite un código más limpio, fácil de entender y mantener, cumpliendo además con límites de complejidad cognitiva.

### **Anti-patrón:**

Arrow Code (anidación profunda).

### **Patrón aplicado (Dos alternativas):**

Uso de Guard Clauses.

### **Código corregido:**

```
public String processUser(String user) {  
  
    if (user == null || user.isEmpty()) return "UNKNOWN";  
  
  
  
  
  
  
  
  
  
    if (!user.contains("admin")) return "USER";  
  
  
  
  
  
  
  
  
  
    if (user.contains("test") && user.contains("prod")) return "ADMIN_TEST_PROD";  
  
  
    if (user.contains("test")) return "ADMIN_TEST";  
  
  
  
  
  
  
  
  
  
    return "ADMIN";  
  
}
```

También podría aplicarse Strategy Pattern, pero no es necesario.

## MEDIUM 11

```
40         return "UNKNOWN";
41     }
42
43     // DEFECTO 4: Excepción no capturada
44     public void addUser(String username) {
45         int id = Integer.parseInt(username); // ¿Y si username no es un número?
46
47         users.add(username);
48     }
49
50     // DEFECTO 5: Método nunca usado
51     public int calculateUnusedMetric(int a, int b) {
52         return a + b;
53     }
54 }
```

Remove this useless assignment to local variable "id".

El código contiene una excepción no controlada y un método innecesario. Esto puede provocar fallos en ejecución y aumenta la complejidad del sistema. La solución implementa manejo de excepciones adecuado y elimina código no utilizado. Esto mejora la estabilidad, claridad y mantenimiento del sistema.

### Anti-patrón:

Excepción no controlada y método sin uso.

Patrón aplicado (Dos alternativas):

Uso de Exception Handling Pattern y eliminación de código muerto.

### Código corregido:

```
public void addUser(String username) {
    try {
        Integer.parseInt(username);
    } catch (NumberFormatException e) {
```



```
logger.error("Username no es numérico");

}

users.add(username);

}
```

Alternativamente, validación previa con Guard Clauses.

## MEDIUM 12

```
50 public int calculateUnusedMetric(int a, int b) {
51     return a + b;
52 }
53
54 // DEFECTO 6: Resource leak (recurso no cerrado)
55 public String readUserFile(String path) throws Exception {
56
57     java.nio.file.Files.lines(java.nio.file.Paths.get(path))
58         .forEach(System.out::println);
59
60     return "File read"; // Stream no se cierra explícitamente
61 }
62
63 public List<String> getUsers() {
64     return users;
65 }
```

Replace generic exceptions with specific library exceptions or a custom exception.

Replace this use of System.out by a logger.

Replace this use of System.out by a logger.

Se presenta un problema de fuga de recursos al no cerrar correctamente un stream, además del uso de excepciones genéricas y salida por consola. La solución utiliza try-with-resources, asegurando el cierre automático de recursos. También mejora el manejo de errores y logging, logrando un código más seguro, eficiente y profesional.

## Anti-patrón:



Resource leak, uso de excepción genérica y System.out.

### Patrón aplicado (Dos alternativas):

Uso de Try-With-Resources.

### Código corregido:

```
public String readUserFile(String path) throws IOException {  
  
    try (Stream<String> lines = Files.lines(Paths.get(path))) {  
  
        lines.forEach(logger::info);  
  
    }  
  
    return "File read";  
  
}
```

También podría aplicarse Facade Pattern para manejo de archivos, pero es innecesario aquí.

### MEDIUM 13



El código repite problemas de manejo de recursos, excepciones genéricas y logging inadecuado. Esto puede generar fallos y dificultar el mantenimiento. La solución aplica try-with-



resources y uso de logger, mejorando la gestión de recursos y la trazabilidad. El resultado es un código más robusto, limpio y alineado con buenas prácticas.

Anti-patrón:

Recurso no cerrado + excepción genérica + logging incorrecto.

Patrón aplicado (Dos alternativas):

Aplicación de Try-With-Resources + Logger.

**Código corregido:**

```
public String readUserFile(String path) throws IOException {  
  
    try (Stream<String> lines = Files.lines(Paths.get(path))) {  
  
        lines.forEach(logger::info);  
  
    }  
  
    return "File read";  
  
}
```

Alternativamente, uso de Service Layer Pattern para encapsular lectura de archivos, pero no es necesario en este caso.

Issue Low

Low 1



```
3 public class BankAccount {
4     private double balance;
5     private String accountNumber;
6
7     // DEFECTO 7: Campo mutable sin sincronización
8     public double balance_public = 0.0;
9
10
11
12
13
14
15
16
17
18 public BankAccount(String accountNumber, double initialBalance) {
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
264
```



## Patrón aplicado

Encapsulación (Information Hiding)

Código corregido:

```
private double balancePublic = 0.0;

public double getBalancePublic() {

    return balancePublic;

}

public void setBalancePublic(double balancePublic) {

    this.balancePublic = balancePublic;

}
```

Defecto 2: El método deposit presenta una validación incorrecta al permitir valores negativos o no manejar adecuadamente este caso. Esto puede generar inconsistencias en el saldo de la cuenta, afectando la lógica del sistema. La ausencia de una validación clara permite operaciones inválidas que comprometen la integridad de los datos.

## Anti-patrón

Lógica defectuosa / Validación incorrecta

## Patrón aplicado

Guard Clauses (validación temprana)





Código corregido:

```
public void deposit(double amount) {  
  
    if (amount <= 0) {  
  
        throw new IllegalArgumentException("El monto debe ser mayor que cero");  
  
    }  
  
    balance += amount;  
  
}
```

## Low 2

```
public class BankAccount {  
    private double balance;  
    private String accountNumber;  
  
    // DEFECTO 7: Campo mutable sin sincronización  
    public double balance_public = 0.0;
```

Make balance\_public a static final constant or non-public and provide accessors if needed.

Rename this field "balance\_public" to match the regular expression `^[a-z][a-zA-Z0-9]*$`.

El atributo `balance_public` no cumple con las convenciones de nomenclatura de Java, ya que utiliza `snake_case` en lugar de `camelCase`. Según los estándares, los nombres de variables deben iniciar en minúscula y continuar con letras o números sin guiones bajos. Esto afecta la legibilidad y consistencia del código, dificultando su mantenimiento en equipos de desarrollo.



## Anti-patrón

Bad Naming (Convenciones de nombres incorrectas)

## Patrón aplicado

Naming Conventions (Clean Code)

## Código corregido:

```
private double balancePublic = 0.0;
```

## low 3

```
return this.accountNumber == other.accountNumber; // Incorrecto
}

// DEFECTO 10: Código muerto
public double getBalance() {
    double temp = balance * 2;

    Remove this unused "temp" local variable.

    return balance; // temp nunca se usa
}
}
```

En el método getBalance, se declara una variable local temp que nunca es utilizada. Esto constituye código muerto, ya que no aporta ninguna funcionalidad al programa. La presencia de variables innecesarias afecta la claridad del código, puede confundir a otros desarrolladores y dificulta el mantenimiento. Además, incrementa innecesariamente la complejidad del método.

## Anti-patrón

Dead Code (Código muerto)

## Patrón aplicado



Clean Code (eliminación de código innecesario)

**Código corregido:**

```
public double getBalance() {  
  
    return balance;  
  
}
```

**Low 4**

```
// DEFECTO 1: Variable nunca usada  
private int unused_variable = 0;
```

Rename this field "unused\_variable" to match the regular expression `^[a-z][a-zA-Z0-9]*$`.

```
// DEFECTO 2: SQL Injection (vulnerabilidad)  
public boolean validateUser(String username) {  
    String query = "SELECT * FROM users WHERE name = '" + username + "'";  
}
```

Remove this unused "query" local variable.

El atributo `unused_variable` no cumple con las convenciones de nomenclatura de Java, ya que utiliza `snake_case` en lugar de `camelCase`. Según los estándares, los nombres de variables deben iniciar en minúscula y continuar con letras o números sin guiones bajos. Esto afecta la legibilidad, consistencia y mantenibilidad del código dentro del proyecto.

**Anti-patrón**

Bad Naming (Convenciones de nombres incorrectas)

**Patrón aplicado**

Naming Conventions (Clean Code)



## Código corregido:

```
private int unusedVariable = 0;
```

low 5



En el método validateUser, se declara la variable query, pero nunca se utiliza. Esto constituye código muerto, ya que no aporta ninguna funcionalidad. Además, la consulta está construida de forma insegura mediante concatenación de strings, lo que podría generar vulnerabilidades como SQL Injection si se utilizara. La presencia de código innecesario afecta la claridad y mantenibilidad del sistema.

## Anti-patrón

Dead Code (Código muerto)

(Relacionado) Construcción insegura de consultas

Patrón aplicado

Clean Code (eliminación de código innecesario)



(Recomendado) Uso de consultas parametrizadas

**Código corregido:**

```
public boolean validateUser(String username) {  
  
    if (username == null || username.isEmpty()) {  
  
        return false;  
  
    }  
  
    return users.contains(username);  
  
}
```

Low 6



En el método validateUser, se declara la variable query, pero no se utiliza en ninguna operación posterior. Esto constituye código muerto, ya que no aporta funcionalidad al sistema. Además, la consulta está construida mediante concatenación de cadenas, lo que representa un



riesgo potencial de seguridad (SQL Injection). La presencia de este código innecesario afecta la claridad, mantenibilidad y calidad del código.

Anti-patrón

Dead Code (Código muerto)

Patrón aplicado

Clean Code (eliminación de código innecesario)

(Adicional) Validación simple sin uso de SQL

**Código corregido:**

```
public boolean validateUser(String username) {  
  
    if (username == null || username.isEmpty()) {  
  
        return false;  
  
    }  
  
    return users.contains(username);  
  
}
```

Low 7



```
public String processUser(String user) {  
    if (user != null) {  
        if (user.length() > 0) {
```

Use "isEmpty()" to check whether a "String" is empty or not.

```
        if (user.contains("admin")) {  
            if (user.contains("test")) {  
                if (user.contains("prod")) {  
                    return "ADMIN_TEST_PROD";  
                } else {  
                    return "ADMIN_TEST";  
                }  
            } else {  
                return "ADMIN";  
            }  
        }  
    }  
}
```

En el método processUser, se utiliza `user.length() > 0` para validar si un String está vacío. Aunque funcional, esta práctica no es la recomendada en Java, ya que reduce la legibilidad del código. El uso de `isEmpty()` es más claro, expresivo y alineado con las buenas prácticas. Este tipo de detalle afecta la calidad del código y su mantenibilidad, especialmente en equipos de desarrollo.

## Anti-patrón

Bad Practice / Poor Readability

## Patrón aplicado

Clean Code (uso de métodos expresivos)

## Código corregido:

```
public String processUser(String user) {
```

```
    if (user != null && !user.isEmpty()) {
```





```
if (user.contains("admin")) {  
  
    if (user.contains("test")) {  
  
        if (user.contains("prod")) {  
  
            return "ADMIN_TEST_PROD";  
  
        } else {  
  
            return "ADMIN_TEST";  
  
        }  
  
    } else {  
  
        return "ADMIN";  
  
    }  
  
} else {  
  
    return "USER";  
  
}  
  
}  
  
return "UNKNOWN";  
  
}
```

## Low 8

```
0         return "UNKNOWN";
1     }
2
3     // DEFECTO 4: Excepción no capturada
4     public void addUser(String username) {
5         int id = Integer.parseInt(username); // ¿Y si username no es un número?
6
7         users.add(username);
8     }
9
10    // DEFECTO 5: Método nunca usado
11    public int calculateUnusedMetric(int a, int b) {
12        return a + b;
13    }
```

Remove this unused "id" local variable.

En el método `addUser`, se declara la variable `id` mediante `Integer.parseInt(username)`, pero nunca se utiliza posteriormente. Esto constituye código muerto, ya que no aporta funcionalidad al sistema. Además, esta conversión puede lanzar una excepción si el valor no es numérico, lo que introduce un riesgo innecesario. La presencia de variables sin uso afecta la claridad, mantenibilidad y calidad del código.

## Anti-patrón

Dead Code (Código muerto)

## Patrón aplicado

Clean Code (eliminación de código innecesario)

## Código corregido:

```
public void addUser(String username) {
    users.add(username);
}
```



}

## Conclusiones

Evidenciamos una falta de la aplicación de buenas prácticas de codificación, lo que generaba problemas como código poco legible, difícil de mantener y con estructuras innecesariamente complejas. Esto afectando directamente la calidad del software.

Presentaba una alta complejidad cognitiva, especialmente por el uso de anidaciones profundas (Arrow Code), lo que requería la necesidad de aplicar técnicas de refactorización como Guard Clauses, que simplifican el flujo lógico y mejoran la comprensión del código.

Presenciamos deficiencias importantes en términos de seguridad, como la vulnerabilidad a SQL Injection, lo que indica que no se están aplicando prácticas seguras en el manejo de datos. El uso de consultas parametrizadas es fundamental para evitar estos riesgos

Se encontraron errores lógico-semánticos los cuales son peligrosos porque el código compila correctamente, pero se comporta de forma inesperada en tiempo de ejecución. No los detecta el compilador sino herramientas de análisis estático como SonarQube o pruebas unitarias.

Al no aplicar programación defensiva. Al no validar entradas del usuario se generan vulnerabilidades como SQL Injection y excepciones no controladas que comprometen la estabilidad y seguridad del sistema.

Defectos de mantenibilidad: No rompen la funcionalidad del programa, pero incrementan la deuda técnica dificultando la lectura, modificación y escalabilidad del código a largo plazo.